Investigação de Dados Aninhados (JSON) no Kibana

1. Visão Geral

Em análise de dados e cibersegurança, é comum encontrar informações estruturadas de forma aninhada (**Nested Objects**), como no formato **JSON**. Esta documentação detalha como realizar buscas granulares dentro desses objetos utilizando a sintaxe nativa do Kibana (**KQL - Kibana Query Language**), permitindo a extração de evidências sem a necessidade de ferramentas externas.

2. Estrutura de Dados Aninhada

Para esta análise, o foco reside no campo `comments` (comentários), que é uma matriz de objetos composta pelos subcampos:

- **author**: Identifica o analista ou utilizador responsável pelo comentário.
- **text**: Contém a descrição técnica ou observação do evento.

3. Metodologia de Consulta (Sintaxe KQL)

As consultas aninhadas permitem filtrar registos com base em propriedades específicas dentro da matriz de objetos.

3.1. Identificação de Campos Existentes

Para retornar todos os registos que possuam qualquer valor preenchido no subcampo de autor, utiliza-se o caractere curinga (`*`):

- **Query:** `comments.author:*`

3.2. Filtro por Valor Exato

Para isolar registos associados a um autor específico (Exemplo: Alice):

- **Query:** `comments.author:"Alice"`

3.3. Consultas Compostas (Operadores Lógicos)

Para aumentar a precisão da busca combinando critérios de diferentes subcampos, utiliza-se o operador `AND`:

- **Query:** `comments.author:"Alice" AND comments.text:attack`
- **Objetivo:** Localizar comentários escritos por "Alice" que contenham especificamente a palavra "ataque" (*attack*).

---

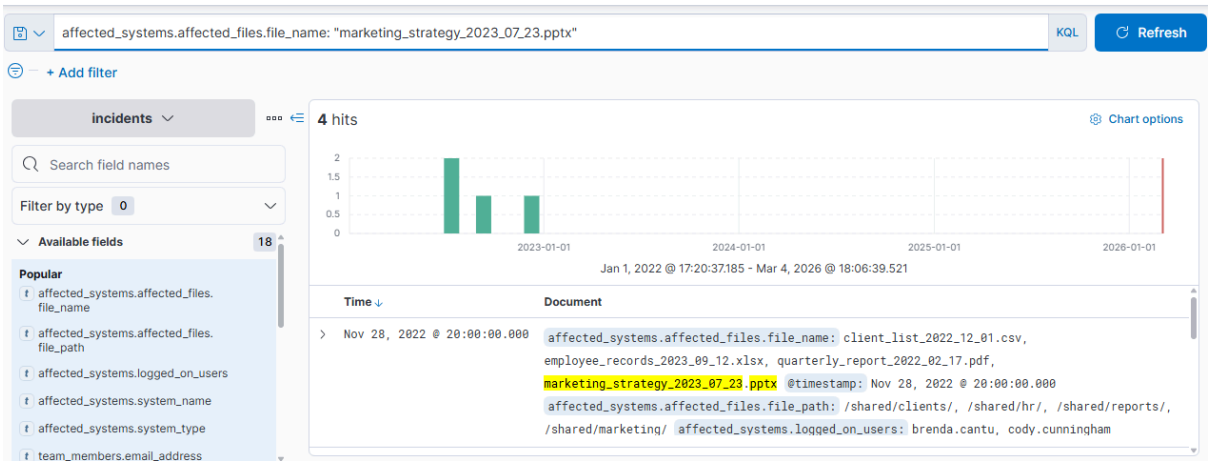
4. Resolução de Casos Práticos (Tarefas de Investigação)

Abaixo seguem os resultados obtidos através da aplicação das técnicas de filtragem no conjunto de dados `incidents`.

Questão 1: Localização de Arquivo Específico

Pergunta: Quantos incidentes existem em que o arquivo afetado é `marketing_strategy_2023_07_23.pptx`?

- **Filtro Aplicado:**affected_systems.affected_files.file_name: `"marketing_strategy_2023_07_23.pptx"`
- **Resposta:**

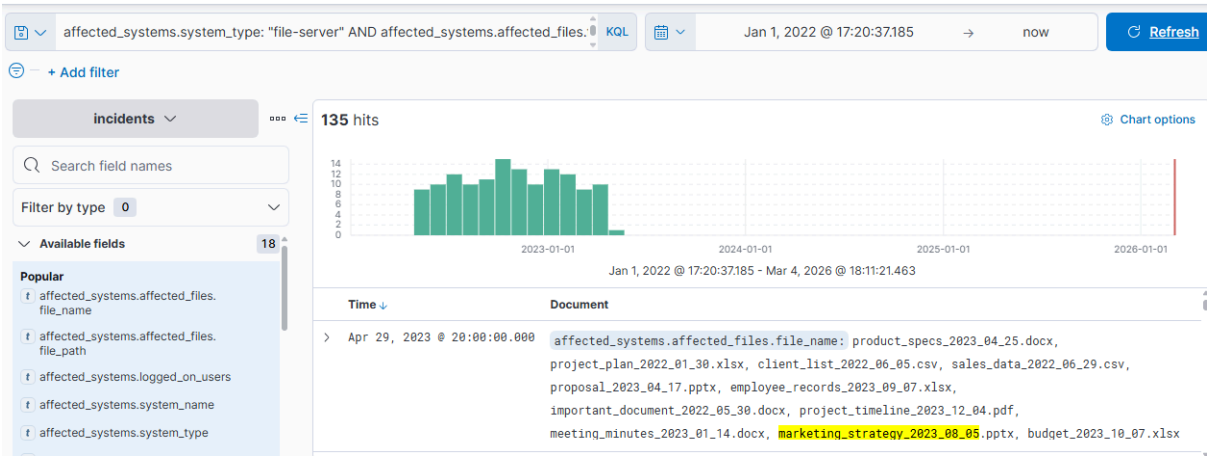


Questão 2: Busca por Padrão em Servidores de Arquivos

Pergunta: Quantos incidentes existem em que os arquivos afetados nos servidores de arquivos têm o título `marketing_strategy`?

- **Filtro Aplicado:** `systems_affected: "file-server" AND affected_files: *marketing_strategy*`

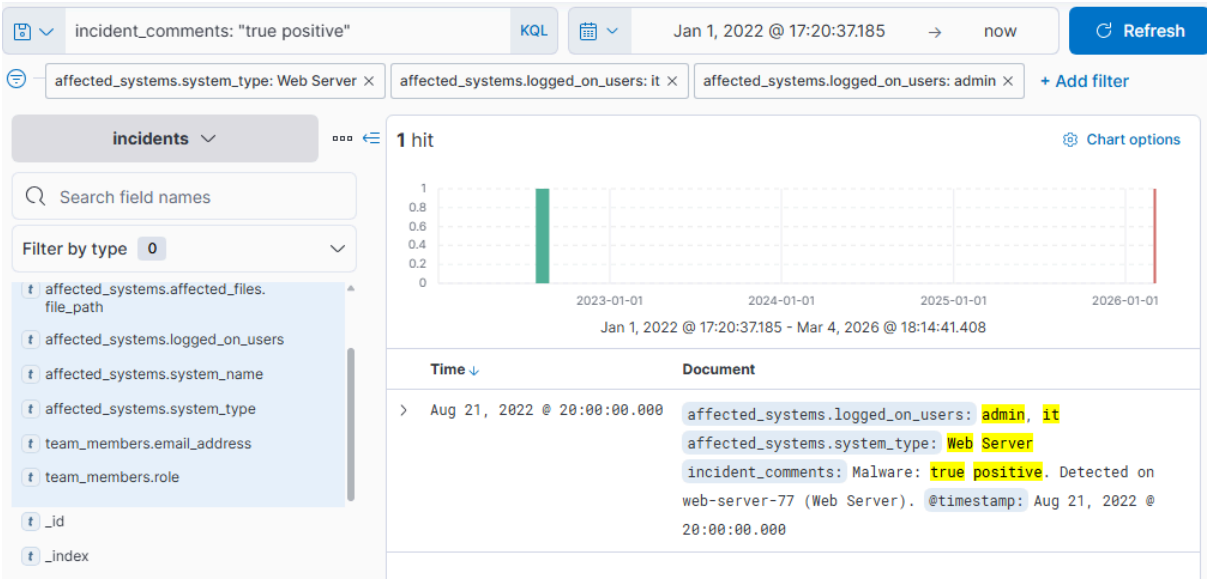
- **Resposta:**



Questão 3: Identificação de Servidor (Verdadeiro Positivo)

Pergunta: Há um alerta de verdadeiro positivo em um servidor web no qual o administrador e os usuários estavam conectados. Qual é o nome do servidor web?

- **Metodologia:** Filtragem por `type: "web-server"` combinada com a análise de logs de autenticação para identificar sessões simultâneas de `admin` e utilizadores comuns no mesmo hostname.
- **Resposta:**



Aqui está a documentação estruturada para o seu **Google Docs**, focada em consultas por intervalo e filtros numéricos/temporais no Kibana.

Relatório Técnico: Consultas por Intervalo (Range Queries) no Kibana

1. Visão Geral

As consultas por intervalo permitem filtrar documentos que possuem valores de campo dentro de um limite especificado. Esta funcionalidade é essencial para analisar métricas de desempenho (como tempos de resposta), níveis de severidade e janelas temporais de eventos.

2. Operadores de Comparação

Para definir os intervalos, utilizamos operadores matemáticos padrão na barra de pesquisa do Kibana (KQL):

- **>=**: Maior ou igual a.
- **<=**: Menor ou igual a.
- **>**: Maior que.
- **<**: Menor que.

Exemplo de Conjunto de Dados (Alertas)

ID do Alerta	Tipo de Alerta	Tempo de Resposta (segundos)
1	Deteção de Malware	120
2	Tentativa de login incomum	240
3	Trânsito suspeito	600
4	Acesso não autorizado	300
5	E-mail de phishing	180

3. Construção de Consultas

3.1. Filtros Numéricos Simples

Para procurar alertas com tempo de resposta superior a 100 segundos:

- **Query:** response_time_seconds >= 100

3.2. Filtros Combinados (Intervalos Fechados)

Para isolar valores entre 100 e 300 segundos, utilizamos o operador lógico **AND**:

- **Query:** response_time_seconds >= 100 AND response_time_seconds < 300
- **Resultado:** Documentos com ID 1, 2 e 5.

3.3. Filtros Temporais (@timestamp)

Os intervalos são frequentemente aplicados a datas e horas. Exemplos de sintaxe:

- **Data e Hora Completa:** @timestamp < "yyyy-MM-ddTHH:mm:ssZ"
- **Apenas Data:** @timestamp > yyyy-MM-dd

---

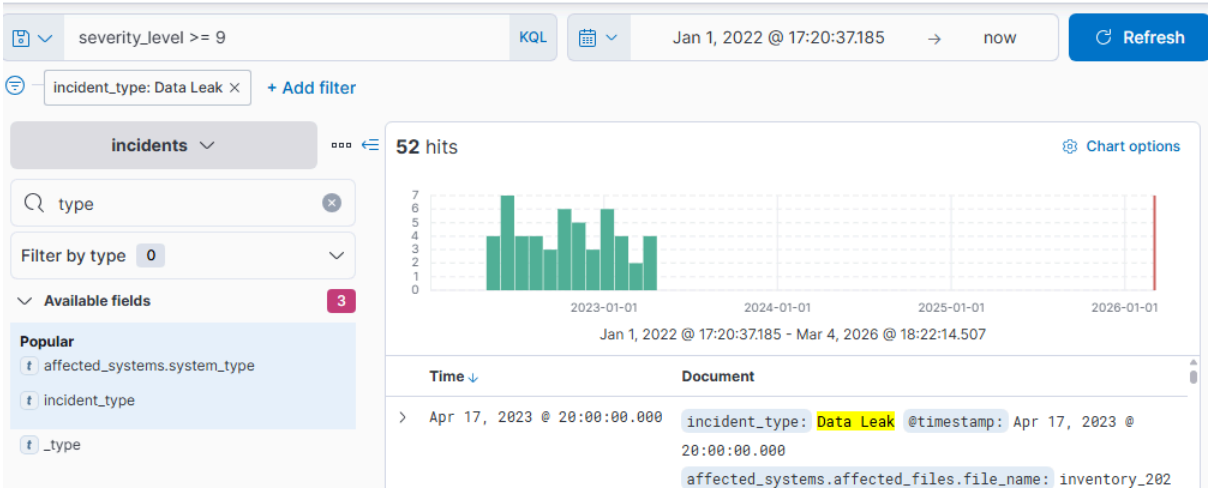
4. Resolução de Casos Práticos (Tarefas de Investigação)

Utilizando o conjunto de dados incidents (1000 entradas), aplicamos as técnicas de intervalo para responder às seguintes questões de segurança:

Questão 1: Incidentes de Alta Gravidade

Pergunta: Quantos incidentes de "Vazamento de Dados" têm um nível de gravidade igual ou superior a 9?

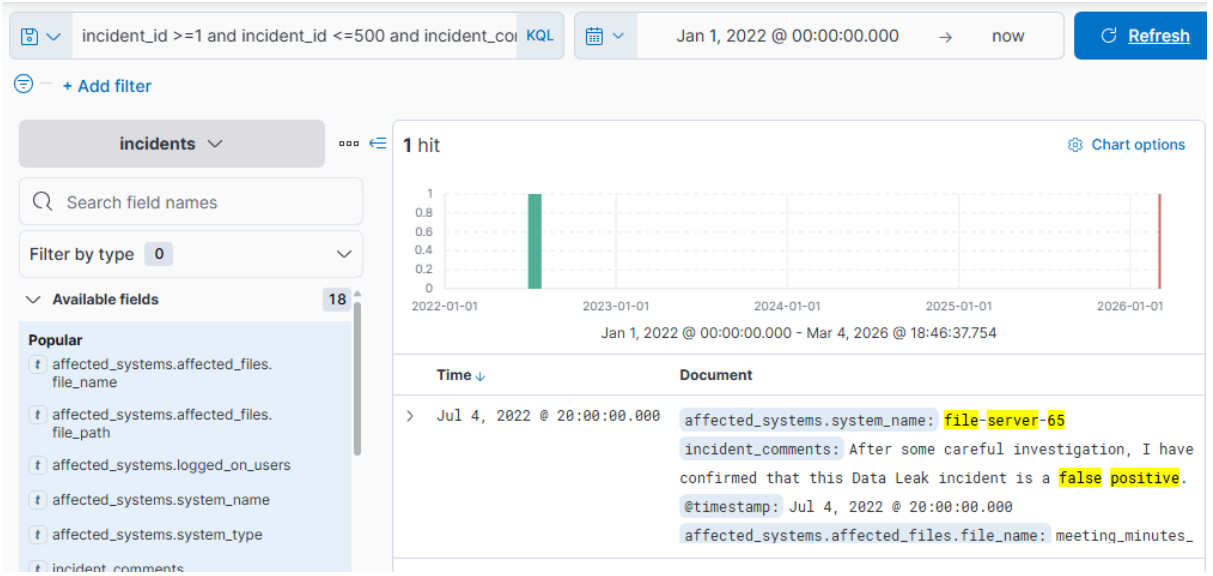
- **Filtro Aplicado:** incident_type: "Data Leak" AND severity >= 9
- **Resposta:**



Questão 2: Identificação de Analista em Incidente Específico

Pergunta: Dos incidentes com IDs de 1 a 500, qual é o endereço de e-mail do analista do SOC que comentou no incidente que o vazamento de dados no servidor de arquivos 65 era um falso positivo?

- **Metodologia:** 1. Filtrar pelo intervalo de IDs: incident_id >= 1 AND incident_id <= 500.
  2. Adicionar termos de busca para o conteúdo do comentário e sistema: comments.text: "false positive" AND systems_affected: "file-server-65".
  3. Identificar o campo analyst.email ou equivalente no documento resultante.
- **Resposta:**



Aqui está a documentação técnica estruturada para o seu **Google Docs**, focada na funcionalidade de Busca Aproximada (Fuzzy Search) utilizando a sintaxe Lucene no Kibana.

Relatório Técnico: Implementação de Busca Aproximada (Fuzzy Search) no Kibana

1. Visão Geral

A busca aproximada (**Fuzzy Search**) é uma técnica essencial para localizar documentos que contenham variações ortográficas, erros de digitação ou inconsistências de dados. Ela permite que o motor de busca recupere termos que sejam semelhantes ao termo pesquisado, baseando-se em um valor de imprecisão (*fuzziness*).

2. Mecanismo de Funcionamento e Sintaxe

O mecanismo utiliza o caractere til (~) para indicar o nível de aproximação permitido (distância de edição de caracteres).

- **Sintaxe básica:** `nome_do_campo:termo_de_busca~valor_de_aproximação`
- **Valor de Aproximação:** Define quantos caracteres podem ser diferentes (inseridos, deletados ou substituídos).

Exemplo de Aplicação

Ao pesquisar por um servidor cujo nome pode ter sido digitado incorretamente como "serber":

- **Query:** `host_name:server01~1`
- **Resultados possíveis:** `server01`, `serber01`.
- **Com imprecisão 2:** `host_name:server01~2` pode retornar também `sorvor01`.

3. Configuração do Ambiente: Mudança para Lucene

Para utilizar buscas aproximadas avançadas e funcionalidades de *boosting*, é necessário alterar o sistema de sintaxe do Kibana de **KQL** para **Lucene**.

Procedimento:

1. Na barra de pesquisa do Kibana, localize o botão **KQL**.
2. No menu suspenso, alterne a opção **Kibana Query Language** de "On" para **"Off"**.
3. A partir deste momento, o Kibana processará as consultas usando a sintaxe Lucene.

---

4. Resolução de Casos Práticos (Investigação de Erros Ortográficos)

Utilizando o conjunto de dados `incidents`, analisamos o desempenho do analista **JLim** e a consistência de seus registros.

Caso de Estudo: Erros na palavra "true"

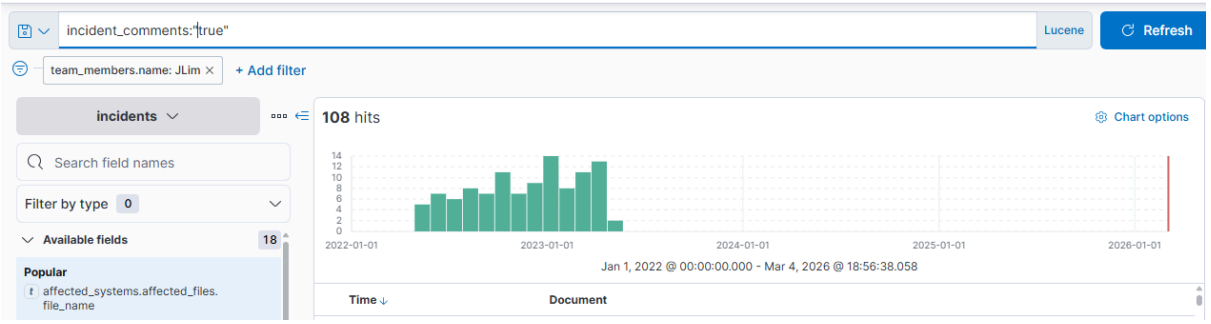
Pergunta: Incluindo os erros ortográficos, em quantos casos o analista **JLim** escreveu a palavra "true" incorretamente?

Para resolver esta questão, comparamos a busca exata com a busca aproximada:

Etapa A: Busca Exata (Sem Aproximação)

Nesta etapa, filtramos apenas os casos onde a palavra foi escrita de forma correta ou exatamente conforme o termo.

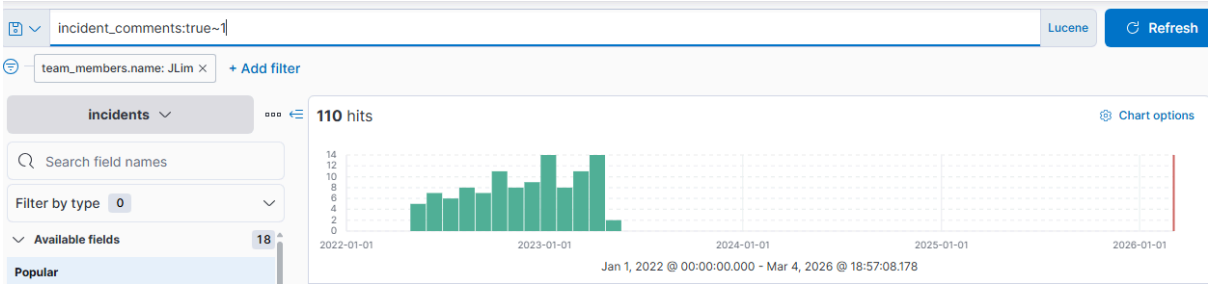
- **Query Lucene:** `analyst:"JLim" AND incident_comments:"true"`



Etapa B: Busca Aproximada (Com Fuzziness)

Nesta etapa, aplicamos o operador ~ para capturar variações como "treu", "tru" ou "trrue".

- **Query Lucene:** `analyst:"JLim" AND incident_comments:true~1`



Conclusão da Tarefa: A diferença entre os resultados da Etapa A e da Etapa B revela o volume de incidentes registrados com erros de digitação pelo analista.

---

5. Limitações Técnicas

Embora poderosa, a busca aproximada possui restrições importantes:

1. **Dados Aninhados:** Não funciona em campos de matrizes de objetos (Nested JSON).
2. **Sequências:** Só encontra correspondências em sequências de uma única palavra.
3. **Performance:** Valores de imprecisão muito altos podem aumentar o tempo de resposta em índices muito extensos.

Aqui está a documentação técnica estruturada para o seu **Google Docs**, focada em **Buscas por Proximidade (Proximity Queries)** utilizando a sintaxe Lucene no Kibana.

Relatório Técnico: Implementação de Buscas por Proximidade no Kibana

1. Visão Geral

A busca por proximidade permite recuperar documentos onde dois ou mais termos aparecem dentro de uma distância específica entre si em um campo de texto. Diferente da busca exata de frases, esta técnica oferece flexibilidade para encontrar palavras relacionadas que podem estar separadas por outros termos ou em ordens diferentes.

2. Conceito de Slop e Sintaxe

O parâmetro de inclinação (**Slop**) define a distância máxima de "movimentos" ou posições que os termos podem ter para serem considerados uma correspondência.

- **Sintaxe Lucene:** `nome_do_campo:"termo1 termo2"~valor_slop`
- **Diferenciação Crítica:** * **Busca Aproximada (Fuzzy):** Aplica-se a uma *única palavra* (`termo~1`).
    - **Busca por Proximidade:** Aplica-se a uma *frase entre aspas* (`"termo1 termo2"~1`).

Exemplo de Aplicação

Para encontrar mensagens onde "server" e "error" estão próximos no campo `log_message`:

- **Query:** `log_message:"server error"~1`
- **Correspondências:** * "**Server error**: failed login" (distância 0).
    - "**Server**: Detected **error**" (distância 1 - a palavra 'Detected' está entre elas).

3. Consultas Complexas e Operadores

A busca por proximidade pode ser combinada com operadores lógicos (`AND`, `OR`) para cenários de investigação multifacetados.

- **Exemplo:** `log_message:"server error"~1 OR "login server"~1`
- **Objetivo:** Capturar variações de falhas de servidor ou problemas de login que ocorram na mesma vizinhança de texto.

---

4. Resolução de Casos Práticos (Análise de Incidentes)

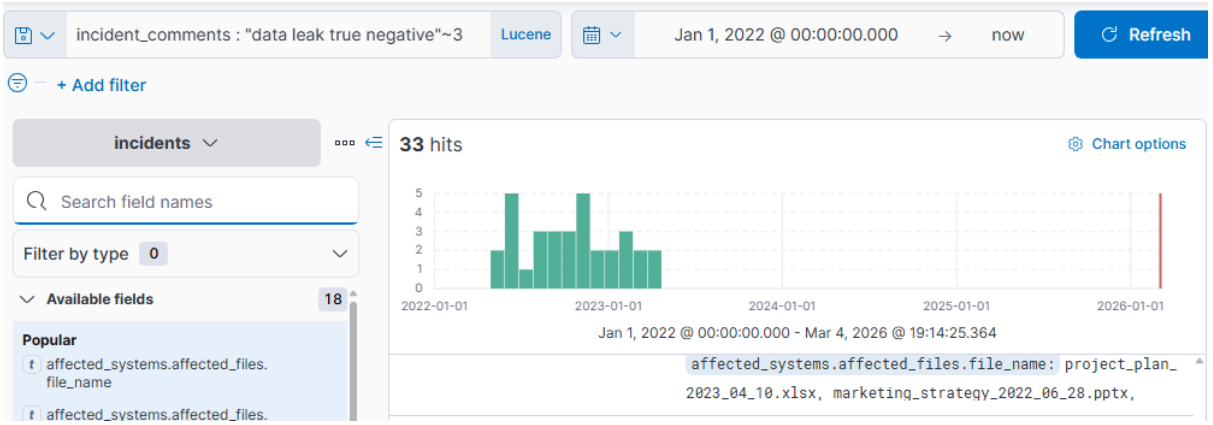
Utilizando o conjunto de dados `incidents` e a sintaxe **Lucene** (KQL desativado), realizamos uma busca avançada nos comentários dos analistas.

Caso de Estudo: Correlação de Termos nos Comentários

Pergunta: Quantos incidentes existem quando se procura pelas palavras "data leak" e "true negative" nos comentários, onde haja pelo menos 3 palavras de distância entre elas?

Procedimento de Investigação:

1. **Configuração:** Garantir que o índice selecionado é o `incidents` e que a sintaxe está definida como **Lucene**.
2. **Construção da Query:**
    - Para buscar a frase com uma distância de até 3 palavras:
    - **Query Lucene:**incident_comments :`"data leak true negative"~3`



Análise do Resultado: Esta consulta é vital para identificar casos onde o analista confirmou que uma suspeita de vazamento de dados foi, na verdade, um falso alarme (falso positivo), mesmo que a explicação no texto seja extensa.

Relatório Técnico: Uso de Expressões Regulares (Regexp) no Kibana

1. Visão Geral

Expressões Regulares (Regexp) são padrões poderosos utilizados para encontrar correspondências complexas em valores de campos. No Kibana, o uso de Regexp permite realizar buscas que ultrapassam as limitações de strings simples ou caracteres curingas,

sendo essencial para identificar padrões de ataques, URLs maliciosas ou variações de telemetria.

2. Sintaxe e Operadores Básicos

Para que uma consulta seja interpretada como Expressão Regular no Kibana, o padrão deve ser obrigatoriamente envolvido por barras invertidas (`/`).

- **Sintaxe:** `campo:/padrão/`
- **Exemplos de Operadores:**
  - `.`: Corresponde a qualquer caractere individual.
  - `*`: Corresponde a zero ou mais ocorrências do caractere anterior.
  - `|`: Operador de alternância (OU lógico). Ex: `(S|M)` corresponde a termos que começam com S ou M.

3. Comportamento por Tipo de Dado

O comportamento da busca Regexp no Kibana varia drasticamente conforme o mapeamento interno do campo:

3.1. Campos do tipo "keyword"

Nestes campos, a string é tratada como um bloco único. A expressão regular é comparada com o valor total do campo.

- **Exemplo:** `Event_Type:/(S|M).*/` retornará apenas entradas onde o campo completo comece com S ou M.

3.2. Campos do tipo "text"

Campos de texto são **tokenizados** (divididos em palavras individuais). A expressão regular é comparada contra cada palavra (*token*) separadamente.

- **Comportamento:** Ao buscar `Description:/(s|m).*/`, o Kibana destacará palavras individuais como "SQL", "service" ou "steal", em vez da frase completa.

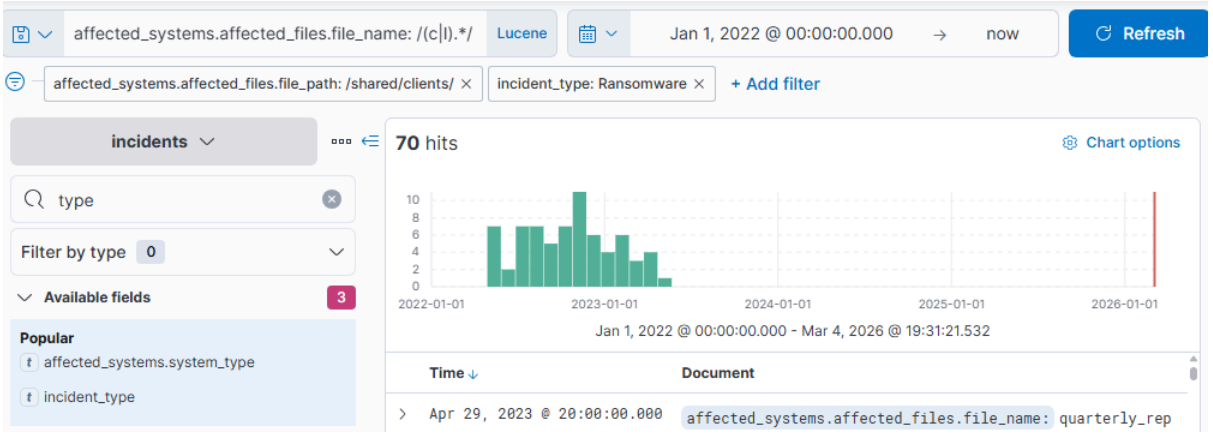
---

4. Resolução de Casos Práticos (Investigação de Ransomware)

Utilizando o conjunto de dados `incidents` e a sintaxe **Lucene**, aplicamos Regex para identificar um incidente específico de sequestro de dados.

Caso de Estudo: Arquivos de Clientes e Ransomware

Pergunta: Quantos casos ocorreram em que um arquivo "client_list" foi afetado por ransomware?

Procedimento de Investigação:

1. **Configuração:** Seleção do índice `incidents` e ativação da sintaxe **Lucene** (KQL Off).
2. **Construção da Query:** * Precisamos combinar a detecção do termo "ransomware" com o padrão do arquivo "client_list" (que pode ter extensões ou datas variadas no nome).
    - **Query Lucene:** `type:/ransomware/ AND affected_files:/client_list.*/`